COURSEWORK ASSIGNMENT SUBMISSION

Name: Daksh Srivastava **Roll No:** 150096725087
Subject: Software Engineering & Project Management **Day:** Day 1 — Introduction to Software Engineering

---

Section 7: Mini Design Challenge — Build the Idea Before the Code

Project Context: Conceptual design of the College Event Management System (CEMS) without writing code, focusing on software engineering principles, stakeholder identification, requirement analysis, risk management, and verification planning.

Step 1: Stakeholder Identification

Stakeholder Group	Primary Goal	Key Operational Concern
Students	Browse scheduled campus events and register seamlessly within 3 clicks.	Seat availability uncertainty and duplicate registration errors.
Event Coordinators	Track real-time attendee counts and enforce seat capacity limits.	Manual attendance reconciliation and overbooking issues.
Faculty Approvers	Review event details, venue safety, and approve budgets before publication.	Late submissions and untracked administrative changes.
Volunteers / Desk Staff	Quickly verify registered attendees at the entry gate using student roll numbers.	Slow gate verification queues due to unresponsive systems.
College Administrators	Generate centralized reports on resource utilization and student participation.	Data privacy breaches and lack of audit logging.

Step 2: Functional Requirements

- **FR-01 (Event Discovery):** The system shall display a list of all active events filtered by category (Technical, Cultural, Sports) and date.
- **FR-02 (Registration):** The system shall allow authenticated students to register for an open event if current registrations are strictly below maximum capacity.
- **FR-03 (Cancellation):** The system shall allow registered students to cancel their registration up to 24 hours prior to the event scheduled start time.
- **FR-04 (Confirmation):** The system shall generate a unique 8-character alphanumeric confirmation token immediately upon successful registration.
- **FR-05 (Attendance Report):** The system shall allow authorized coordinators to export a PDF/CSV attendance manifest containing student roll numbers and timestamps.

Step 3: Non-Functional Requirements

- **NFR-01 (Performance & Latency):** Under a concurrent load of 500 active users, the event catalog page shall load within 1.8 seconds for the 95th percentile (p95) of HTTP requests.
- **NFR-02 (Security & Access Control):** All administrative endpoints (event creation, approval, and attendee export) must enforce Role-Based Access Control (RBAC) via signed JWT tokens, rejecting unauthorized requests with HTTP 403 Forbidden.

Step 4: Risk Identification & Mitigation

Identified Risk	Severity / Likelihood	Prevention / Mitigation Measure
Duplicate Registrations	High / Medium	Apply a composite unique database index on (student_id, event_id) and enforce database transaction isolation.
Traffic Surge at Registration Open	High / High	Deploy an in-memory Redis rate limiter and cache static event metadata on an edge CDN.
Unauthorized Student Data Access	Critical / Low	Mask sensitive student contact details in public views and mandate SSL/TLS encryption for all data in transit.

Step 5: Test Case Specification

- **Test Case ID:** TC-CEMS-001 (Boundary Condition: Event Capacity Limit)
- **Starting Condition (Precondition):** Event "Hackathon 2026" has a capacity limit of 100 participants. Current registered count is exactly 100.
- **Action / Trigger:** Student with ID 150096725087 clicks "Register Now" and submits the registration form.
- **Expected Result:** System rejects registration, displays an error message "Event is full. Registration closed.", and does not increment attendee count.
- **Verification Evidence:** Database record count remains 100; HTTP response code is 409 Conflict; user screen displays expected error banner.

Section 7: Challenge Question — Late Change Investigation

Challenge Question: If the college asks for payment collection one day before launch, what must be investigated before committing to delivery?

Investigation Checklist & Impact Assessment:

- **1. Technical & Gateway Integration Feasibility:** A reliable payment gateway (e.g., Razorpay, Stripe) requires merchant account verification, webhook handlers, and cryptographic signature verification. Integrating this in 24 hours without thorough sandbox testing risks transaction loss.

- **2. Database Schema & State Transitions:** A new payment state machine is required (Initiated, Pending, Captured, Failed, Refunded). Modifying the schema and registration logic 24 hours before launch bypasses regression testing.
- **3. Legal, Reconciliation & Refund Policies:** Financial transactions require institutional compliance, merchant KYC approval, audit logging, and automated refund handling for failed registrations or event cancellations.
- **Recommendation:** Reject immediate deployment for Day 1 launch. Recommend launching Version 1.0 with free registration or offline fee collection at the desk, and schedule digital payment integration for Version 1.1 following a structured 2-week sprint.

Section 11: Homework — Observe Software Like an Engineer

Case Study: Real-world engineering analysis of Google Pay (UPI Mobile Application).

- **1. Problem Solved:** Eliminates cash dependency by enabling instantaneous, bank-to-bank electronic fund transfers via the NPCI UPI unified protocol using mobile phone numbers or QR codes.
- **2. Target Users & Objectives:** Retail consumers (peer-to-peer transfers, utility bill payments) and merchants (instant payment acceptance, daily settlement reconciliation).
- **3. Three Core Functional Features:** (a) Dynamic and static QR code scanning; (b) Inter-bank fund transfer using recipient UPI VPA; (c) Real-time account balance enquiry protected by UPI PIN.
- **4. Two Critical Quality Attributes:** (a) Security (end-to-end device binding, encrypted PIN payloads, biometric authentication); (b) Reliability / Fault Tolerance (two-phase commit with banking switches to prevent un-reconciled debit states).
- **5. Failure Scenario & Impact:** Network timeout occurring after user account debit but before NPCI confirmation. Impact: User experiences funds deduction without merchant receipt. System must log pending state and trigger automated reverse-clearing within T+1 banking hours.
- **6. SDLC Activity (Testing Phase):** Conducting chaos testing and network latency injection to verify that transaction state machines gracefully handle packet drops without dropping transaction journal entries.
- **7. Proposed Improvement & Verification:** Implement an automated offline queue for merchant notification confirmation using local soundbox or BLE beacons. Verification: Measure confirmation latency in low-bandwidth (<2G) conditions.
- **Preparation for Next Class:** Stable Requirements Example: Core Banking Ledger System (regulatory accounting rules change infrequently). Changing Requirements Example: Campus Social & Event App (user expectations, UI preferences, and feature sets evolve with student feedback).